

PAYMENT TRACK, DAY 3: invoices, FRONTEND

Angular 16 + Material, Midnight theme. **2 new components, 1 new service, 2 files replaced (model and module), small edits in 4 existing files.** Do the Day 3 backend guide first.

0. What the user sees

- **After paying:** the "Payment was successful!" pop-up says "Your invoice is ready and opens next". Pressing OK opens the **invoice page** instead of My Enrollments.
- **Invoice page:** seller name, address and GSTIN, invoice number, date, payment reference, "Billed to", one row per course, then the totals (with the GST lines when GST is on) and a **Download PDF** button.
- **My Invoices** (new menu link): a table of all the customer's invoices with **View** and **PDF** buttons.
- **My Enrollments:** each order has an **Invoice** button.
- An order with no invoice, or someone else's invoice, shows a friendly "There is no invoice for this order" instead of an error.

Honest status. Real browser, pretend backend, stand-in Razorpay pop-up: **39 checks, 0 failures, 0 page errors.** They cover: pay and land on the invoice, the number, three course lines, the GST split (before tax 2,158.47 and tax 388.53 on a 2,547.00 cart), seller details, the PDF actually downloaded as `invoice-INV-2026-000001.pdf` and starting with `%PDF-`, My Invoices, the Invoice button on My Enrollments, another customer being refused, and the Day 2 earnings and payouts screens. The pretend backend returns a sample PDF; the real PDF comes from your Spring code.

1. Edits and new files

1. `constant.ts` : add one line inside the `API` object

```
INVOICE: '/api/invoice',
```

`angularapp/src/app/models/billing.model.ts` (ts)

````REPLACE the Day 2 file (it now also has the invoice types) */\* Money is always a whole number of paise (100 paise = 1 rupee). Divide by 100 to show rupees. /*

```
export interface InvoiceLine { courseId: number; title: string; paise: number; }
```

```
export interface Invoice { id: number; invoiceNumber: string; paymentId: number; orderId: number | null; razorpayPaymentId: string | null; customerName: string; customerEmail: string; issuedAt: string; lines: InvoiceLine[]; taxablePaise: number; taxPaise: number; totalPaise: number; gstPercent: number; sellerName: string; sellerAddress: string; sellerGstin: string; }
```

```
export interface EarningRow { id: number; courseId: number; courseTitle: string; paymentId: number; grossPaise: number; feePaise: number; ownerPaise: number; status: 'PENDING' | 'PAID_OUT'; createdAt: string; }
```

```
export interface PayoutView { id: number; instructorUserId: number; instructorName: string; amountPaise: number; earningsCount: number; reference: string; paidByEmail: string; paidAt: string; }
```

```
export interface EarningsSummary { totalPaise: number; pendingPaise: number; paidOutPaise: number; commissionPercent: number; earnings: EarningRow[]; payouts: PayoutView[]; }
```

```
export interface PayableRow { instructorUserId: number; name: string; email: string; mobile: string; pendingPaise: number; pendingCount: number; }
```

```
`angularapp/src/app/services/invoice.service.ts` (ts)

```new file
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { API, API_URL } from '../constant';
import { Invoice } from '../models/billing.model';

/** Invoices of paid orders: list, read, and download as a PDF file. */
@Injectable({ providedIn: 'root' })
export class InvoiceService {
  private apiUrl = API_URL;

  constructor(private http: HttpClient) {}

  mine(): Observable<Invoice[]> {
    return this.http.get<Invoice[]>(`${this.apiUrl}${API.INVOICE}/my`);
  }

  byId(id: number): Observable<Invoice> {
    return this.http.get<Invoice>(`${this.apiUrl}${API.INVOICE}/${id}`);
  }

  byOrder(orderId: number): Observable<Invoice> {
    return this.http.get<Invoice>(`${this.apiUrl}${API.INVOICE}/by-order/${orderId}`);
  }

  /** The PDF travels with the login token (the interceptor adds it), so it is fetched as a blob and then saved. */
  downloadPdf(invoice: Invoice): Observable<Blob> {
    return this.http.get(`${this.apiUrl}${API.INVOICE}/${invoice.id}/pdf`, { responseType: 'blob' });
  }

  /** Saves a blob as a file with the given name. */
  saveBlob(blob: Blob, fileName: string): void {
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = fileName;
    document.body.appendChild(a);
    a.click();
    a.remove();
    setTimeout(() => URL.revokeObjectURL(url), 1000);
  }
}
}
```

angularapp/src/app/components/my-invoices/my-invoices.component.ts (ts)

```
```new file import { Component, OnInit } from '@angular/core'; import { Invoice } from
'../models/billing.model'; import { InvoiceService } from '../services/invoice.service';
```

```

/* A customer's invoices, newest first, each with View and Download PDF. / @Component({ selector: 'app-
my-invoices', templateUrl: './my-invoices.component.html', styleUrls: ['./my-invoices.component.css'] })
export class MyInvoicesComponent implements OnInit { invoices: Invoice[] = []; loaded = false; loadError =
"; downloadError = "; busyId: number | null = null;

```

```

constructor(private api: InvoiceService) {}

```

```

ngOnInit(): void { this.api.mine().subscribe({ next: list => { this.invoices = list; this.loaded = true; }, error: err
=> { this.loaded = true; this.loadError = err.status === 404 ? 'Complete your profile first, then your invoices
will appear here.' : 'Could not load your invoices. Please try again.'; }); }

```

```

download(i: Invoice): void { this.busyId = i.id; this.downloadError = ""; this.api.downloadPdf(i).subscribe({
next: blob => { this.api.saveBlob(blob, 'invoice-' + i.invoiceNumber + '.pdf'); this.busyId = null; }, error: () =>
{ this.busyId = null; this.downloadError = 'The PDF could not be downloaded. Please try again.'; }); }

```

```

trackById(_: number, i: Invoice): number { return i.id; } }

```

```

`angularapp/src/app/components/my-invoices/my-invoices.component.html` (html)

```new file
<div class="page">
  <div class="page-head">
    <div>
      <h2>My invoices</h2>
      <p>One invoice for every payment you made. Download it as a PDF any time.</p>
    </div>
  </div>

  <div class="form-error" role="alert" id="invoicesError" *ngIf="loadError"><mat-icon>error_outline</mat-
icon>{{ loadError }}</div>
  <div class="form-error" role="alert" id="downloadError" *ngIf="downloadError"><mat-
icon>error_outline</mat-icon>{{ downloadError }}</div>
  <p class="empty" id="noInvoices" *ngIf="loaded && !loadError && !invoices.length">
    You have no invoices yet. An invoice appears here after you pay for a course. <a routerLink="/customer-
view-courses">Browse courses</a>
  </p>

  <div class="table-card" *ngIf="invoices.length">
    <table class="data">
      <thead><tr><th>Invoice</th><th>Date</th><th>Courses</th><th class="num">Total</th><th></th></tr>
</thead>
      <tbody>
        <tr *ngFor="let i of invoices; trackBy: trackById" [attr.id]="'invoice' + i.id">
          <td><strong>{{ i.invoiceNumber }}</strong></td>
          <td>{{ i.issuedAt | date: 'd MMM y' }}</td>
          <td>{{ i.lines.length }} {{ i.lines.length === 1 ? 'course' : 'courses' }}</td>
          <td class="num">{{ i.totalPaise / 100 | currency:'INR':'symbol':'1.2-2' }}</td>
          <td class="acts">
            <a mat-stroked-button color="primary" class="view" [routerLink]="['/invoice', i.id]">View</a>
            <button mat-flat-button color="primary" class="solid pdf" type="button" (click)="download(i)"
[disabled]="busyId === i.id">
              <mat-icon>download</mat-icon>{{ busyId === i.id ? 'Preparing...' : 'PDF' }}
            </button>
          </td>
        </tr>
      </tbody>
    </table>
  </div>
</div>

```

angularapp/src/app/components/my-invoices/my-invoices.component.css (CSS)

```
```new file / Customer: my invoices (Midnight) / .empty { padding: 40px 0; text-align: center; color: var(--muted); } .acts { display: flex; gap: 8px; justify-content: flex-end; flex-wrap: wrap; } .num { text-align: right; } @media (max-width: 640px) { .acts { justify-content: flex-start; } }
```

```
```angularapp/src/app/components/invoice-view/invoice-view.component.ts``` (ts)

```new file
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { Invoice } from '../../models/billing.model';
import { InvoiceService } from '../../services/invoice.service';

/**
 * One invoice on screen. Reached as /invoice/:id (from My invoices) or /invoice/order/:orderId
 * (right after paying, or from an order). The PDF button downloads the same invoice as a file.
 */
@Component({
 selector: 'app-invoice-view',
 templateUrl: './invoice-view.component.html',
 styleUrls: ['./invoice-view.component.css']
})
export class InvoiceViewComponent implements OnInit {
 invoice: Invoice | null = null;
 loaded = false;
 notFound = false;
 loadError = '';
 downloadError = '';
 busy = false;

 constructor(private route: ActivatedRoute, private api: InvoiceService) {}

 ngOnInit(): void {
 const orderId = this.route.snapshot.paramMap.get('orderId');
 const id = this.route.snapshot.paramMap.get('id');
 const call = orderId ? this.api.byOrder(Number(orderId)) : this.api.byId(Number(id));
 call.subscribe({
 next: inv => { this.invoice = inv; this.loaded = true; },
 error: err => {
 this.loaded = true;
 if (err.status === 404) { this.notFound = true; }
 else { this.loadError = 'Could not load this invoice. Please try again.'; }
 }
 });
 }

 get addressLines(): string[] {
 return (this.invoice?.sellerAddress || '').split('\n').map(s => s.trim()).filter(s => !!s);
 }

 download(): void {
 if (!this.invoice) { return; }
 const inv = this.invoice;
 this.busy = true;
 this.downloadError = '';
 this.api.downloadPdf(inv).subscribe({
 next: blob => { this.api.saveBlob(blob, 'invoice-' + inv.invoiceNumber + '.pdf'); this.busy = false; },
 error: () => { this.busy = false; this.downloadError = 'The PDF could not be downloaded. Please try again.'; }
 });
 }
}
```

new file

# Invoice

Keep this for your records.

```
All invoices download{{ busy ? 'Preparing...' : 'Download PDF' }}
error_outline{{ loadError }}
error_outline{{ downloadError }}
```

There is no invoice for this order. Invoices are made for online payments. Back to My Enrollments

{{ invoice.gstPercent > 0 ? 'Tax invoice' : 'Invoice' }}

{{ invoice.sellerName }} {{ l }} GSTIN: {{ invoice.sellerGstin }}

Invoice no.

{{ invoice.invoiceNumber }}

Date

{{ invoice.issuedAt | date: 'd MMM y' }}

Payment ref.

{{ invoice.razorpayPaymentId }}

Billed to {{ invoice.customerName }} {{ invoice.customerEmail }}

| Course        | Amount                                                |
|---------------|-------------------------------------------------------|
| {{ l.title }} | {{ l.paise / 100   currency:'INR':'symbol':'1.2-2' }} |

Amount before tax{{ invoice.taxablePaise / 100 | currency:'INR':'symbol':'1.2-2' }}

GST {{ invoice.gstPercent }}% (included in the price){{ invoice.taxPaise / 100 | currency:'INR':'symbol':'1.2-2' }}

Total paid{{ invoice.totalPaise / 100 | currency:'INR':'symbol':'1.2-2' }}

```
`angularapp/src/app/components/invoice-view/invoice-view.component.css` (css)

new file
/* Customer: one invoice (Midnight) */
.head-actions { display: flex; gap: 10px; flex-wrap: wrap; }
.empty { padding: 40px 0; text-align: center; color: var(--muted); }
.sheet { max-width: 760px; padding: 28px; border-radius: var(--radius); background: var(--surface); border: 1px solid var(--line); box-shadow: var(--shadow); display: flex; flex-direction: column; gap: 22px; }
.sheet header { display: flex; justify-content: space-between; gap: 24px; flex-wrap: wrap; }
.sheet h3 { font-size: 22px; margin: 0 0 8px; }
.seller { display: block; color: var(--ink); }
.line { display: block; color: var(--muted); font-size: 13.5px; }
dl { margin: 0; display: grid; gap: 6px; }
dl div { display: grid; grid-template-columns: 96px 1fr; gap: 10px; }
dt { font: 700 12.5px var(--display); color: var(--gold-text); }
dd { margin: 0; color: var(--ink-2); overflow-wrap: anywhere; }
.billed { display: flex; flex-direction: column; gap: 2px; }
.label { font: 700 12.5px var(--display); color: var(--gold-text); }
.num { text-align: right; }
```

```
.totals { align-self: flex-end; min-width: 300px; display: grid; gap: 6px; }
.totals div { display: flex; justify-content: space-between; gap: 24px; color: var(--ink-2); }
.totals .grand { margin-top: 4px; padding-top: 10px; border-top: 1px solid var(--line); color: var(--ink);
font: 800 19px var(--display); }
@media (max-width: 520px) { .sheet { padding: 18px; } .totals { min-width: 0; width: 100%; } }
```

## angularapp/src/app/modules/billing/billing.module.ts (ts)

```
````REPLACE the Day 2 file (it now also has the three invoice routes) import { NgModule } from
'@angular/core'; import { RouterModule, Routes } from '@angular/router'; import { SharedModule } from
'../shared/shared.module'; import { AuthGuard } from '../components/authguard/authguard.guard';
import { ROLES } from '../constant'; import { INSTRUCTOR_ROLE } from '../models/instructor.model';
import { MyInvoicesComponent } from '../components/my-invoices/my-invoices.component'; import {
InvoiceViewComponent } from '../components/invoice-view/invoice-view.component'; import {
InstructorEarningsComponent } from '../components/instructor-earnings/instructor-earnings.component';
import { AdminPayoutsComponent } from '../components/admin-payouts/admin-payouts.component';

// 'invoice/order/:orderId' is listed before 'invoice/:id' so that "order" is never read as an id const routes:
Routes = [ { path: 'my-invoices', component: MyInvoicesComponent, canActivate: [AuthGuard], data: {
roles: [ROLES.CUSTOMER] } }, { path: 'invoice/order/:orderId', component: InvoiceViewComponent,
canActivate: [AuthGuard], data: { roles: [ROLES.CUSTOMER, ROLES.ADMIN] } }, { path: 'invoice/:id',
component: InvoiceViewComponent, canActivate: [AuthGuard], data: { roles: [ROLES.CUSTOMER,
ROLES.ADMIN] } }, { path: 'instructor/earnings', component: InstructorEarningsComponent, canActivate:
[AuthGuard], data: { roles: [INSTRUCTOR_ROLE] } }, { path: 'admin-payouts', component:
AdminPayoutsComponent, canActivate: [AuthGuard], data: { roles: [ROLES.ADMIN] } } ];

/* Invoices for customers, earnings for instructors, payouts for admins. * Import it in AppModule
BEFORE AppRoutingModuleModule so these routes are registered before the "" fallback. */ @NgModule({
declarations: [MyInvoicesComponent, InvoiceViewComponent, InstructorEarningsComponent,
AdminPayoutsComponent], imports: [SharedModule, RouterModule.forChild(routes)] }) export class
BillingModule {} ````
```

2. components/navbar/navbar.component.html : one link, in two places

In the **CUSTOMER** block, after the `My Learning` link:

```
<a routerLink="/my-invoices" routerLinkActive="active">My Invoices</a>
```

and in the phone menu, after the customer's `My Learning` link:

```
<a mat-menu-item routerLink="/my-invoices">My Invoices</a>
```

3. components/my-orders/my-orders.component.html : put an Invoice button next to View details

Find this line (inside `<header>` of each order):

```
<a mat-stroked-button color="primary" class="details-link" [routerLink]="['/order-details',
order.orderId]">View details</a>
```

and replace it with:

```
<span class="hd-actions"><a mat-stroked-button color="primary" class="invoice-link" [routerLink]="
['/invoice/order', order.orderId]">Invoice</a><a mat-stroked-button color="primary" class="details-link"
[routerLink]="['/order-details', order.orderId]">View details</a></span>
```

Then add one line at the end of `components/my-orders/my-orders.component.css`:

```
.hd-actions { display: inline-flex; gap: 8px; flex-wrap: wrap; }
```

4. `components/place-order/place-order.component.ts`: four small edits

These make the page open the invoice after a successful payment. If you already replaced this file from CHANGES-7 or Day 1, search for the exact lines below.

Edit 4a. Under the line `paymentDone = false;` add:

```
/** The order saved by a verified Razorpay payment; its invoice is shown after the confirmation. */
private paidOrderId: number | null = null;
```

Edit 4b. In `verifyRazorpay`, change the start of the success handler. Find:

```
next: () => {
  // the server saved the order and emptied the cart
  this.loading = false;
```

replace with:

```
next: (order) => {
  // the server saved the order and emptied the cart
  this.paidOrderId = order?.orderId ?? null;
  this.loading = false;
```

Edit 4c. In `showConfirmation`, change the message text. Find:

```
message: 'You are now enrolled. Find your courses under My Enrollments.'
```

replace with:

```
message: this.paidOrderId ? 'You are now enrolled. Your invoice is ready and opens next.' : 'You are now
enrolled. Find your courses under My Enrollments.'
```

Edit 4d. In `closeConfirmation`, find:

```
this.router.navigate(['/my-orders']);
```

replace with:

```
this.router.navigate(this.paidOrderId ? ['/invoice/order', this.paidOrderId] : ['/my-orders']);
```

(The old no-payment button never sets `paidOrderId`, so it still goes to My Enrollments.)

2. Run it and test it

1. `npm start` : compiles with no red error.
2. Log in as a customer, buy a course with a Razorpay test card. Press OK on the pop-up: the **invoice page** opens. Check the number, the course lines and the total.
3. Press **Download PDF**: a file `invoice-INV-2026-000001.pdf` is saved. Open it.
4. Open **My Invoices**: the invoice is listed; **View** and **PDF** both work.
5. Open **My Enrollments**: the order has an **Invoice** button.
6. Log in as another customer and open `/invoice/1`: you see "There is no invoice for this order", not the first customer's data.

Acceptance checklist

- [] After a test payment, OK opens the invoice page
- [] The invoice shows the number, date, buyer, every course and the total paid
- [] **Download PDF** saves `invoice-INV-....pdf` , and the PDF opens
- [] With `INVOICE_GST_PERCENT=18` the page and the PDF show the GST lines
- [] **My Invoices** lists every payment, newest first
- [] Another customer cannot see it
- [] An instructor has no My Invoices link

3. If something goes wrong (frontend)

| What you see | Why | Fix |
|---|--|--|
| <code>API.INVOICE</code> does not exist | edit 1 missed | add the line to <code>constant.ts</code> |
| Invoice page: "There is no invoice for this order" right after a real payment | the backend is not the Day 3 version, or the backend log shows an invoice error | check the log; open the page again (it makes the invoice if it is missing) |
| Download PDF says "could not be downloaded" | token expired, or the backend is not running | log in again; check the backend |
| The PDF button saves a file named <code>download</code> | the backend header <code>Content-Disposition</code> is missing | use the Day 3 <code>InvoiceController</code> |
| After payment it still goes to My Enrollments | edits 4a to 4d not all applied | apply all four |
| Property <code>'orderId'</code> does not exist in <code>place-order</code> | <code>verifyRazorpayPayment</code> returns <code>Observable<any></code> ; the <code>(order)</code> parameter must stay untyped | keep it as written |